An inode uniquely represents a file object in the file system. Inode is an object internal to the file system and is associated with a file when the file is created. For example, if you create a file *'/root/sandya.txt'*, the file *'sandya.txt'* is created under the parent directory *'/root'* and an inode with an inode number, 100 for instance, is associated with it. In the parent directory *'/root'* a directory entry (dentry) is created with its name as *'sandya.txt'* and the inode number pointed to by the dentry being 100. Now, if you rename the file *'sandya.txt'* to *'lfy.txt'*, what gets changed is only the dentry in the parent directory. Now the dentry in *'/root'* contains the name *'lfy.txt'* with the inode number continuing to point to inode 100. So here is a question for our readers. Most of you would be familiar with the UNIX command *'link'*. Can you describe what happens when the command *'ln –s /root/sandya.txt lfy.txt'* is run?

The inode typically contains information on where the data blocks of a file are located at, the access rights, the creator of the file, the time at which the file was modified/accessed, etc. As discussed before, there is a disk inode representation and an in-memory representation. The structure 'struct inode' defined in <*linux/fs.h*> defines the inode object. Figure 1 contains a simplified definition of the struct inode.

The inode object contains a pointer to the superblock object. It also contains a member field *i_op* which is a pointer to *'struct inode_operations'*. These operations define the implemented operations on the inode object for that specific file system. The structure *'inode_operations'* is defined in <*linux/fs.h*>. The inode operations include *create, lookup, link, unlink, symlink, mkdir, rmdir,* etc. For instance, the *'create'* inode operations are defined as follows:

```
int (*create)(struct inode *dir, struct dentry *dentry, int
mode);
```

The *'create'* inode operation is called from the system call *'create'* and the *'lookup'* operation is called from the system call *'open'*. The *'create'* operation takes as input arguments, the inode corresponding to the parent directory where the file is to be created, the *d-entry* for the file to be created and the creation mode. For instance, if you are creating a file system of type *btrfs*, then the *'i_op'* field will be set suitably so that the *'create'* function pointer in *'inode_operations'* is set to point to *'btrfs_create'*.

D-entry is the other major file system related data structure that describes a directory entry. Recall that VFS treats directories as special types of files. However, when a file system object is looked up through its path, each component of the path is treated as a directory entry. Basically, it will be looked up in its parent directory and if it exists, and is a directory, then follow through further with the next path name component in the path. For example, consider the following path: */root/lfy/sandya.txt*. In this path name, each component, namely, *'/'*, *'root'*, *'lfy'*, and *'sandya.txt'* is a dentry object. Note that a dentry object can be a directory or file. In the earlier example, the components *'/'*,*'root'* and *'lfy'* are all directories whereas *'sandya.txt'* is a file. Dentry objects are represented by the data structure *'struct dentry'* which is defined in <*linux/fs.h*>. One important difference between 'dentry' data structures and other data structures such as superblock and inode, which we discussed earlier, is that there is no disk representation for the 'dentry' object—which is created only in memory and is never made persistent on the disk. Hence, there is no need to write any modifications of the dentry object back to the disk.

```
struct inode

struct list_head i_dentry;
struct timespec i_atime;
struct timespec i_mtime;
struct timespec i_ctime;
gid_t i_gid;
uid_t i_uid;
loff_t i_size;
cost struct file_operations *i_fop;
const struct inode_operations *i_op;
struct address_space *i_mapping;
struct address_space *i_data;
...
```

Figure 1: Struct inode

When a file needs to be accessed, the file system code parses the path name of the file. For instance, given the file */root/lfy/sandya.txt*, the path is parsed in to components, *'/'*, *'root'*, *'lfy'* and *'sandya.txt'*. The file system lookup code checks each component of the path to see if it is a directory and contains the entry corresponding to the next component downstream in the path, until it reaches the last component, which is the actual file we are looking for. Now the question that arises is, how does the file system know where to look for the start of the path name *'/'* or root directory? If you look at the definition of the 'superblock' structure in <*linux/fs.h*>, you will see a field namely *'struct dentry *s_root;'*. When the file system is mounted, this field is initialised with the root directory of the file system, and this is where the lookup operation starts its search for the next component.

Now that we have covered the various important data structures describing the file system metadata, the next question is, how does the file system ensure the consistency of its metadata? Let us consider the following example of creating a file object. This requires two independent operations, namely, creating a directory entry and creating an inode object and associating the directory entry to point to the inode. Suppose, due to power failure, the file system crashes in the middle of these operations so that though we have created the directory entry and inode, we have not updated the directory dentry to point to the inode. When the

Continued on page 71...